

Activity 4

Week 5

6733172621 Patthadon Phengpinij

Collaborators. Claude (for L^AT_EX styling and grammar checking)

Activity: Kubernetes with Multiple Containers

Instructions

1. This assignment is due Wednesday September 9th no later than 11:00 pm on mycourseville (pdf only).
2. Install a kubernetes cluster (using docker desktop or k3d or other distributions).
3. Test your cluster with nginx as followed:
 - a) Deploy an nginx on your cluster using the deployment file from <https://k8s.io/examples/application/deployment.yaml>
 - b) Use kubectl to describe the deployment
4. Deploy a todo service with redis database as followed:
 - a) Study how to create a deployment of multiple containers in a single pod from <https://www.mirantis.com/blog/multi-container-pods-and-container-communication-in-kubernetes/>
 - b) Create a deployment file of a pod with service of the following containers:
 - i) todo (release-2.1)
 - ii) redis

Note: you can use a docker-hub image: natawut/todo-service:release-2.1 and you must ensure that environment for REDIS_HOST is correctly set.
 - c) Deploy the pod, expose service, and show your deployment, service, and pods with kubectl get command.
 - d) Create ingress configuration file, apply the file, and show the ingress information.
 - e) Test the newly deployment:
 - i) Create a new todo record with:

```
1 {"title":"todo-1","detail":"the first todo","duedate":"2022-10-24 11:00:00","tags":[],"completed":false}
```

- ii) Get todo list via localhost:80

Below shows the sample output:

```
1 curl -X POST -d '{"title":"todo-1","detail":"the first todo","duedate":"2024-10-13 11:00:00","tags":[],"completed":false}' http://localhost
```

```
(base) ➔ assignment curl -d '{"title":"todo-1","detail":"the first todo","duedate":"2022-10-24 11:00:00","tags":[],"completed":false}' -w "\n" -X POST http://localhost
{"id":0}
(base) ➔ assignment curl localhost
[{"id":0,"title":"todo-1","detail":"the first todo","completed":false,"duedate":"2022-10-24T11:00:00","tags":[]}]
```

Figure 1: Example output of the todo service.

Note: to help you debug, you may use kubectl logs to check the system log in the container

5. Use the activity grader here:

<https://github.com/fonylew/sds-grader/releases#release-v0.3.3-activity4>

6. Prepare a PDF report containing the following information:
- a) Screenshot when you perform Step 3.
 - b) The deployment file that you create in Step 4.b and ingress file in Step 4.d.
 - c) Screenshot when you perform Step 4.c - Step 4.e.

YOU MUST ALSO USING GRADER (URL is in MCV's assignment)

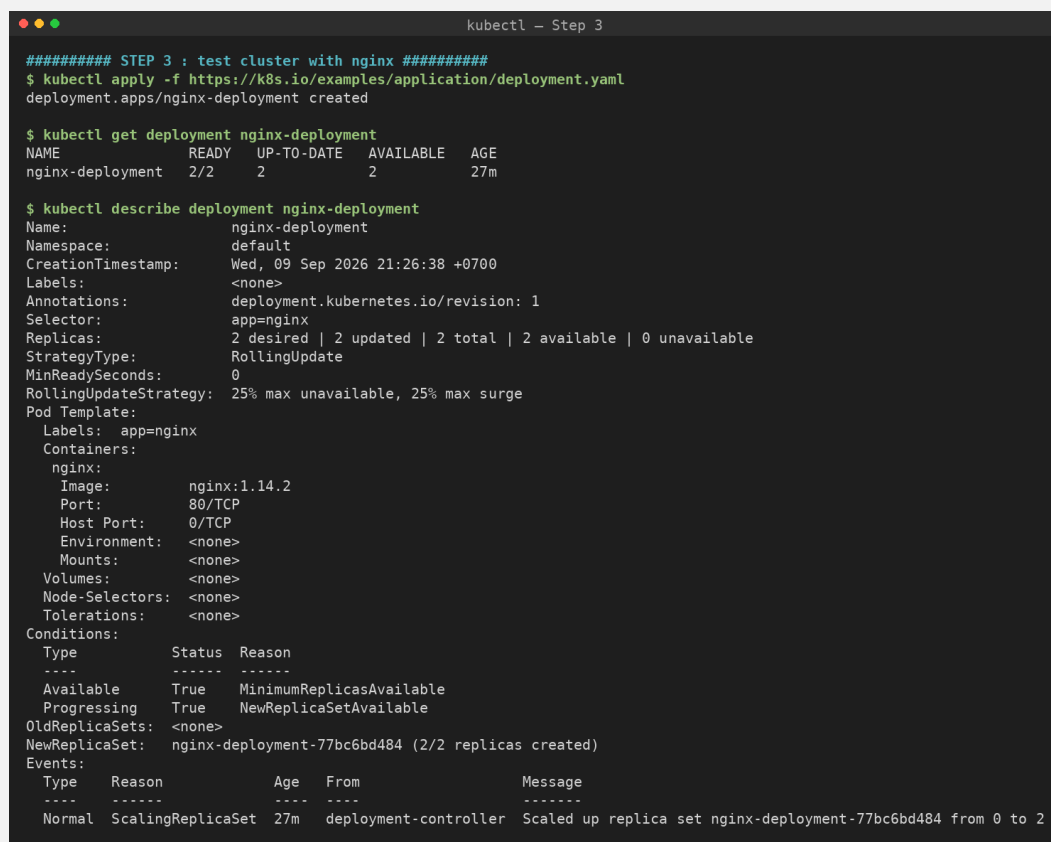
Environment. The cluster is a local single-node **k3d** cluster (k3s in Docker), created with host port 80 mapped to the cluster load balancer so the ingress is reachable at <http://localhost>; k3d ships **Traefik** as the default ingress controller. **kubectl** v1.37 against server v1.35.

```
1 $ k3d cluster create sds --port "80:80@loadbalancer"
2 $ kubectl get nodes
3 NAME                                STATUS    ROLES                  AGE     VERSION
4 k3d-sds-server-0                    Ready    control-plane,master   1m      v1.35.5+k3s1
```

Problem 1. Screenshot when you perform Step 3.

Solution

The example deployment is applied with `kubectl apply -f https://k8s.io/examples/application/deployment.yaml` and then inspected with `kubectl describe deployment nginx-deployment`. It runs 2 replicas of `nginx:1.14.2`, both Available.



```
kubectl - Step 3

##### STEP 3 : test cluster with nginx #####
$ kubectl apply -f https://k8s.io/examples/application/deployment.yaml
deployment.apps/nginx-deployment created

$ kubectl get deployment nginx-deployment
NAME                READY   UP-TO-DATE   AVAILABLE   AGE
nginx-deployment    2/2     2            2           27m

$ kubectl describe deployment nginx-deployment
Name:                nginx-deployment
Namespace:           default
CreationTimestamp:    Wed, 09 Sep 2026 21:26:38 +0700
Labels:              <none>
Annotations:         deployment.kubernetes.io/revision: 1
Selector:             app=nginx
Replicas:            2 desired | 2 updated | 2 total | 2 available | 0 unavailable
StrategyType:        RollingUpdate
MinReadySeconds:     0
RollingUpdateStrategy: 25% max unavailable, 25% max surge
Pod Template:
  Labels:  app=nginx
  Containers:
    nginx:
      Image:   nginx:1.14.2
      Port:    80/TCP
      Host Port: 0/TCP
      Environment: <none>
      Mounts:     <none>
      Volumes:    <none>
      Node-Selectors: <none>
      Tolerations: <none>
  Conditions:
    Type           Status  Reason
    ----           -
    Available      True    MinimumReplicasAvailable
    Progressing    True    NewReplicaSetAvailable
    OldReplicaSets: <none>
    NewReplicaSet:  nginx-deployment-77bc6bd484 (2/2 replicas created)
  Events:
    Type     Reason             Age   From               Message
    ----     -
    Normal   ScalingReplicaSet   27m   deployment-controller   Scaled up replica set nginx-deployment-77bc6bd484 from 0 to 2
```

Figure 2: Step 3 — deploy the example nginx and `kubectl describe deployment nginx-deployment`.

Problem 2. The deployment file that you create in Step 4.b and ingress file in Step 4.d.

Solution

(a) Deployment — `todo-deployment.yaml` (Step 4.b). One pod with two containers, `todo` and `redis`. Containers in the same pod share a network namespace, so the `todo` service reaches `redis` on `localhost` — hence `REDIS_HOST: localhost`.

```
1 apiVersion: apps/v1
2 kind: Deployment
3 metadata:
4   name: todo
5   labels:
```

```

6   app: todo
7 spec:
8   replicas: 1
9   selector:
10    matchLabels:
11     app: todo
12  template:
13    metadata:
14     labels:
15      app: todo
16  spec:
17    containers:
18     - name: todo
19       image: natawut/todo-service:release-2.1
20       imagePullPolicy: IfNotPresent
21       ports:
22        - containerPort: 8000
23       env:
24        - name: REDIS_HOST
25          value: "localhost"      # same pod -> same network namespace
26        - name: REDIS_PORT
27          value: "6379"
28     - name: redis
29       image: redis:7-alpine
30       imagePullPolicy: IfNotPresent
31       ports:
32        - containerPort: 6379
33

```

(b) **Service — todo-service.yaml (Step 4.c, “expose service”)**. A ClusterIP service in front of the todo container’s port 8000. It is *not* exposed on the host, so localhost:8000 and localhost:6379 stay unreachable from outside — only the ingress is public.

```

1  apiVersion: v1
2  kind: Service
3  metadata:
4    name: todo
5    labels:
6     app: todo
7  spec:
8    selector:
9     app: todo
10   ports:
11    - name: http
12      port: 8000
13      targetPort: 8000
14

```

(c) **Ingress — todo-ingress.yaml (Step 4.d)**. Routes http://localhost/ to the todo service. The resource is named todo-ingress (the grader runs `kubectl get ingress` and only accepts the result when the word “ingress” appears in that output).

```

1  apiVersion: networking.k8s.io/v1
2  kind: Ingress
3  metadata:
4    name: todo-ingress
5  spec:
6    ingressClassName: traefik
7    rules:
8     - host: localhost
9       http:
10        paths:
11         - path: /
12           pathType: Prefix
13           backend:
14            service:
15             name: todo
16             port:
17              number: 8000

```

18

Problem 3. Screenshot when you perform Step 4.c - Step 4.e.

Solution

Step 4.c applies `todo-deployment.yaml` and `todo-service.yaml` and shows `kubectl get deployment,svc,pods` — the pod is 2/2 ready (both the `todo` and `redis` containers). Step 4.d applies `todo-ingress.yaml` and shows `kubectl get/describe ingress`. Step 4.e POSTs a `todo` to `http://localhost` (returns `{"id":0}`) and GETs the list back.

```

##### STEP 4c : deployment, service, pods #####
$ kubectl get deployment,svc,pods -l app=todo
NAME                                READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/todo                1/1     1             1           26m

NAME                                TYPE               CLUSTER-IP      EXTERNAL-IP   PORT(S)    AGE
service/todo                        ClusterIP          10.43.50.253    <none>        8000/TCP   26m

NAME                                READY   STATUS    RESTARTS   AGE
pod/todo-6945fd6dc6-plt7h           2/2     Running   0          15m

$ kubectl get pod -l app=todo -o jsonpath (containers in the pod)
todo redis

##### STEP 4d : ingress #####
$ kubectl apply -f todo-ingress.yaml
ingress.networking.k8s.io/todo-ingress created

$ kubectl get ingress
NAME      CLASS    HOSTS      ADDRESS      PORTS    AGE
todo-ingress  traefik  localhost  172.18.0.2   80       15s

$ kubectl describe ingress todo-ingress
Name:          todo-ingress
Labels:        <none>
Namespace:     default
Address:       172.18.0.2
Ingress Class: traefik
Default backend: <default>
Rules:
  Host      Path  Backends
  ----  -
  localhost /    todo:8000 (10.42.0.12:8000)
Annotations: <none>
Events:      <none>

##### STEP 4e : test the deployment #####
$ curl -w '\n' -X POST -d '{"title":"todo-1","detail":"the first todo","duedate":"2022-10-24 11:00:00","tags":[],"completed":false}' http://localhost
{"id":0}

$ curl -w '\n' http://localhost
[{"id":0,"title":"todo-1","detail":"the first todo","completed":false,"duedate":"2022-10-24T11:00:00","tags":[]}]

```

Figure 3: Step 4.c–4.e — `kubectl get deployment,svc,pods`, `describe ingress`, and the `curl` round-trip against `http://localhost`.

Problem 4. Screenshot of the results after running the GRADER.

Solution

The activity4-CEDT grader passes every check — Result: `true` — then prompts for the Student ID and full name to submit the result.

```
activity4-CEDT grader

##### GRADER : activity4-CEDT #####
$ ./activity4-CEDT_linux_amd64
2026/09/09 21:54:01 Current Timestamp: 2026-09-09T21:54:01+07:00
> Enter Kubernetes namespace (leave blank for 'default'):
> Enter domain (leave blank for 'http://localhost'):
2026/09/09 21:54:02 [OK] / Namespace exists and can use kubectl command.
2026/09/09 21:54:02 [OK] / All Kubernetes resources are up and running.
2026/09/09 21:54:02 [OK] / Ingress resource exists in the namespace.
2026/09/09 21:54:02 [OK] / URL http://localhost returns 200.
2026/09/09 21:54:02 [OK] / Todo is up and running at http://localhost
2026/09/09 21:54:02 [OK] / Todo service is inaccessible at http://localhost:8000
2026/09/09 21:54:02 [OK] / Redis service is inaccessible at http://localhost:6379
2026/09/09 21:54:02 [OK] / POST request successful
2026/09/09 21:54:02 [OK] / POST request to http://localhost was successful.
2026/09/09 21:54:02 [OK] / GET request shows result from previous POST request to http://localhost.
2026/09/09 21:54:02 Result: true
2026/09/09 21:54:02 * Looks good! Please enter your StudentID and Full name below
> StudentID: _
    (enter your Student ID and full name here to submit the grader result)
```

Figure 4: activity4-CEDT grader — all checks pass, Result: true.